

Data Structures – Unit 12

Bucharest University of Economic Studies

Graphs



Agenda

- Definition
- Characteristics
- Classification criteria
- Implementations
- Traversing
- Metrics



Graphs - Definition

A graph is a nonlinear structure composed of vertices and links between them, called edges.

- graph $G(V,E)$
- edge = (v, w) , with $v, w \in V$



Graphs - Characteristics

- The vertices are dynamically linked based on the solution requirements;
- The edges mean that two adjacent vertices can be used as a link or path from one point to another;
- The two sets are finite:
 - the set of vertices – each node called also a **vertex**;
 - the set of edges – each edge being a path segment;



Graphs - Characteristics

- elements with multiple links between them;
- visiting each vertex of the graph structure;
- searching for a specific element in the structure;
- identifying a path between two vertices;



Graphs - Classification

- Based on the direction between edges:
 - directed graphs or digraphs;
 - undirected graphs.
- Based on the type of connections between vertices:
 - weighted graphs (each edge has a quantitative significance);
 - unweighted graphs or ordinary graphs (each edge is equally important as any other)



Graphs - Classification

- Based on the existence of edges:
 - connected graphs, there are no unreachable vertices or there is a path between every pair of vertices;
 - disconnected graphs, there is at least one isolated vertex.



Graphs

- A loop is an edge that connects one vertex to itself;
- A path in a graph structure is a sequence of vertices $p_0, p_1, \dots, p_m$, with the property that each pair of adjacent vertices p_i and p_{i+1} are connected by an edge;
- In a digraph the edge between p_i and p_{i+1} has the following significance:
 - p_i is called source;
 - p_{i+1} is called destination;



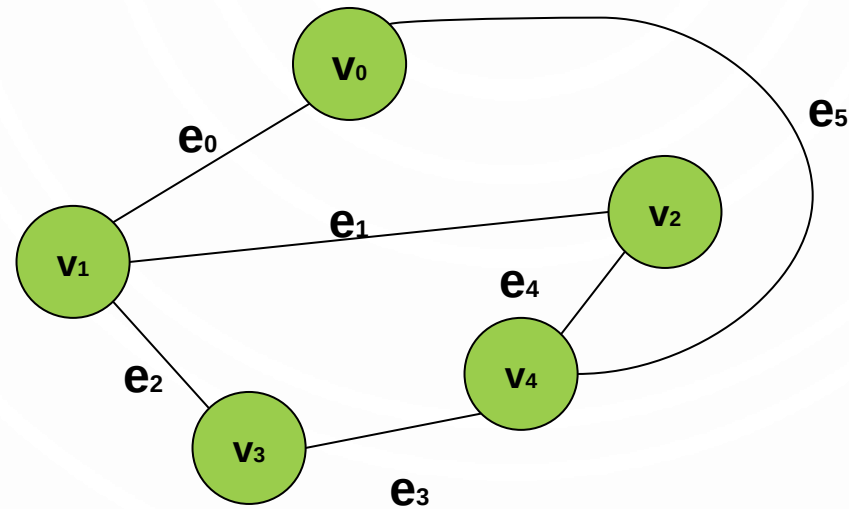
Graphs – Undirected

An undirected graph is a finite set of vertices together with the finite set of edges:

- empty sets $\Rightarrow$ void graph;
- an edge connects two vertices both ways;
- the order of two connected vertices is not important;
- if there is an edge between v and $u \Leftrightarrow$ there is also an edge between u and v ;



Graphs – Undirected



Vertices: v_0, v_1, v_2, v_3, v_4

Edges: $e_0, e_1, e_2, e_3, e_4, e_5$



Digraphs

A directed graph is a finite set of vertices together with the finite set of edges in which the direction is important.

Characteristics:

- two connected vertices have different meanings, one is called source and the other destination;
- The order in which two vertices are connected is important;
- The edge between two vertices u and v $\Leftrightarrow$ the edge connect u source with the v destination;



Graphs - Representation

- Adjacency matrix;
- Edge array or Edge list – edges represented by the two vertices that the edges are incident on;
- An array of simple or double linked lists of adjacent vertices;
- A simple or double linked list of pointers to a simple or double linked lists of adjacent vertices;

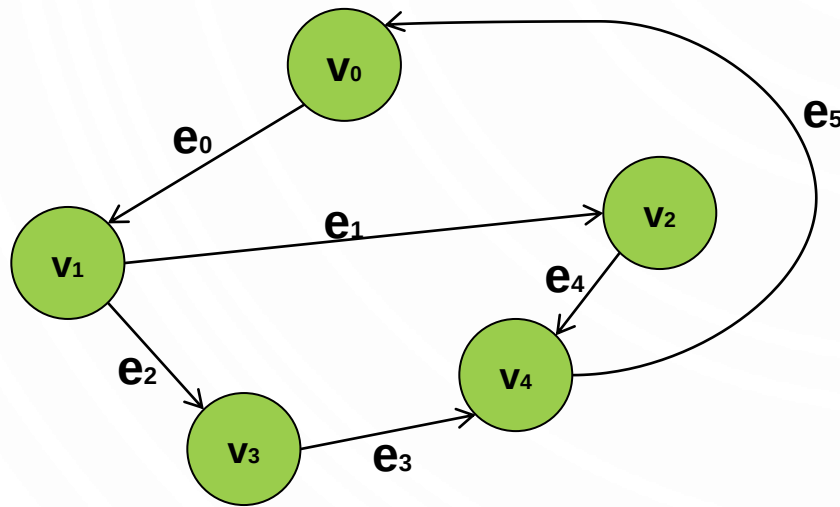


Graphs - Adjacency matrix

- The number of nodes is known;
- The number of edges is also known, which gives us the loading factor for the matrix;
- A square matrix with values:
 - 1 (unweighted graphs);
 - edge's weight (weighted graphs).



Graphs – Adjacency matrix



	0	1	2	3	4
0	false	true	false	false	false
1	false	false	true	true	false
2	false	false	false	false	true
3	false	false	false	false	true
4	true	false	false	false	false

For two different vertices i and j , the matrix value for the i line and j colon is true only if there is an edge between those two, who starts from i and ends in j .

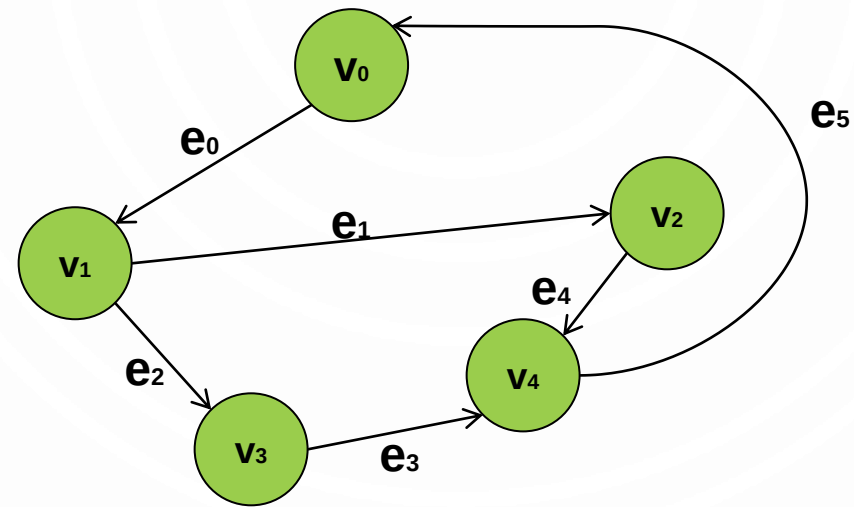


Graphs - Adjacency matrix

- *Pros:* Representation is easier to implement and follow. Removing an edge takes $O(1)$ time. Queries like whether there is an edge from vertex 'u' to vertex 'v' are efficient.
- *Cons:* Consumes more space $O(n^2)$. Even if the graph is sparse(contains less number of edges), it consumes the same space. Adding a vertex is $O(n^2)$ time.



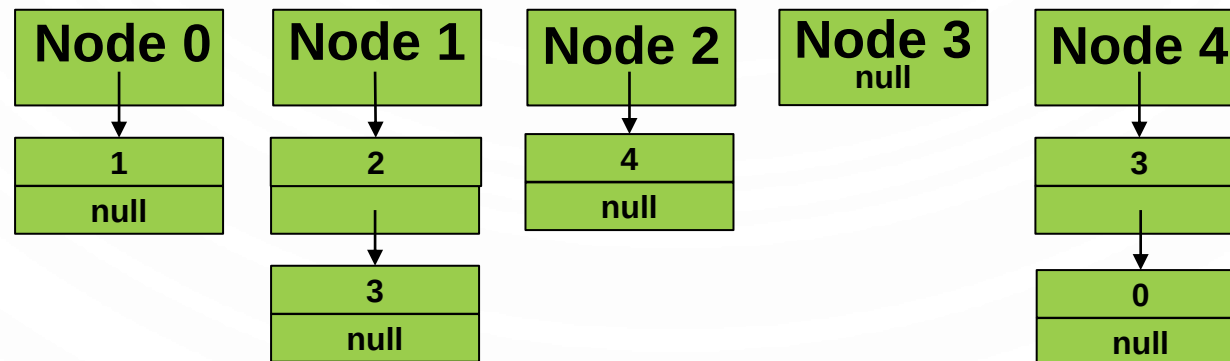
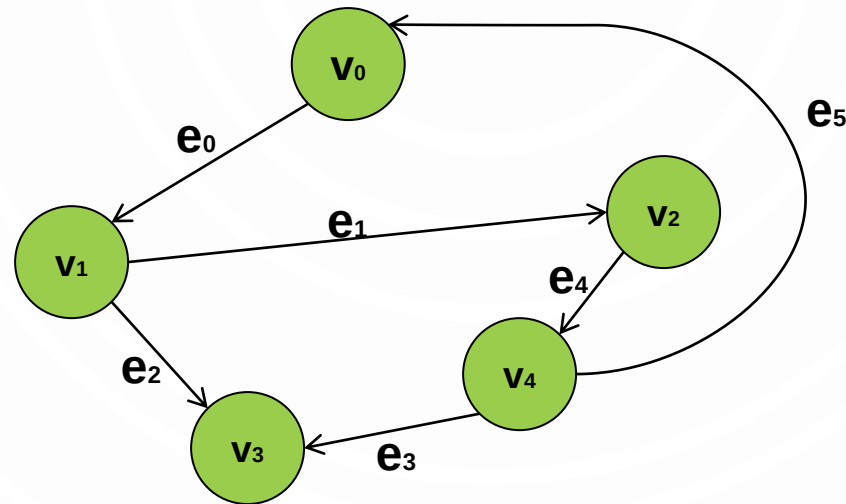
Graphs – Edges array



V_0	V_1	V_1	V_2	V_3	V_4
V_1	V_2	V_3	V_4	V_4	V_0



Graphs – List of lists



Graphs – List of lists

- Network of linked lists of vertices;
- Dynamic maintenance of graph structure;
- Saves space for those vertices that do not have connection;
- Adding an edge is $O(1)$ complexity;
- Removing an edge or queries for the existence of an edge is $O(n)$ complexity;



Graphs – Choosing criteria

- Adding and removing edges and nodes;
- Checking if edges exists;
- Processing at edge level;
- Sparse graphs;



Graphs – Traversals

- The operation has different results based on the starting node;
- The operation must avoid passing through the same node twice by using labels;
- Depth-first traversal;
- Breadth-first traversal.



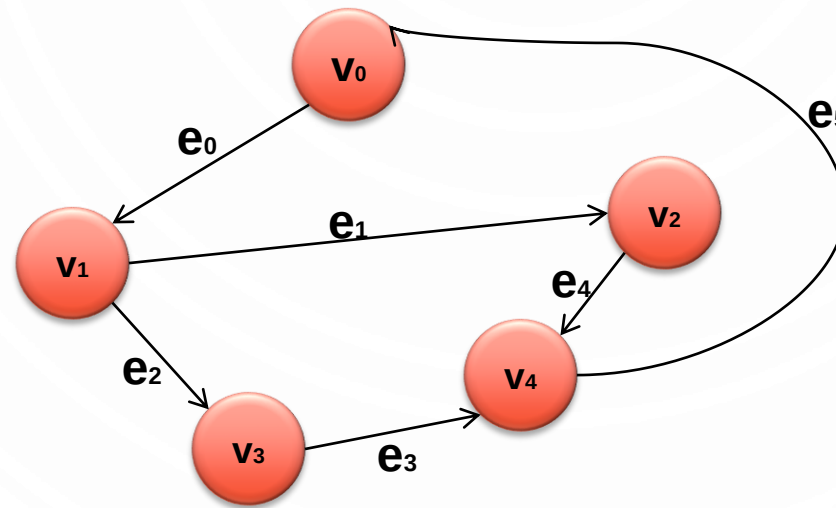
Graphs – Traversing

Depth-first traversal:

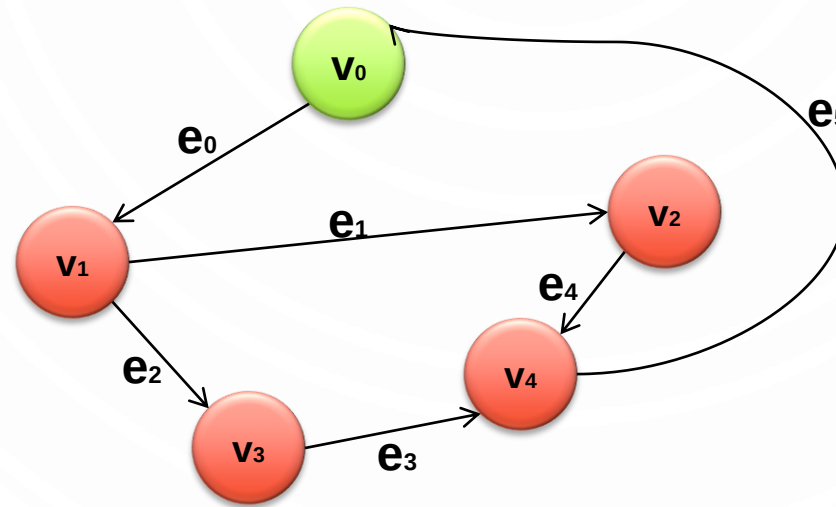
- The equivalent of the preorder traversal of a tree;
- Using additional structures such as: arrays, lists or stacks;
- The operation processes only vertices that are accessible, starting with the head node;
- The visited vertices are labeled as being processed by using a stack;



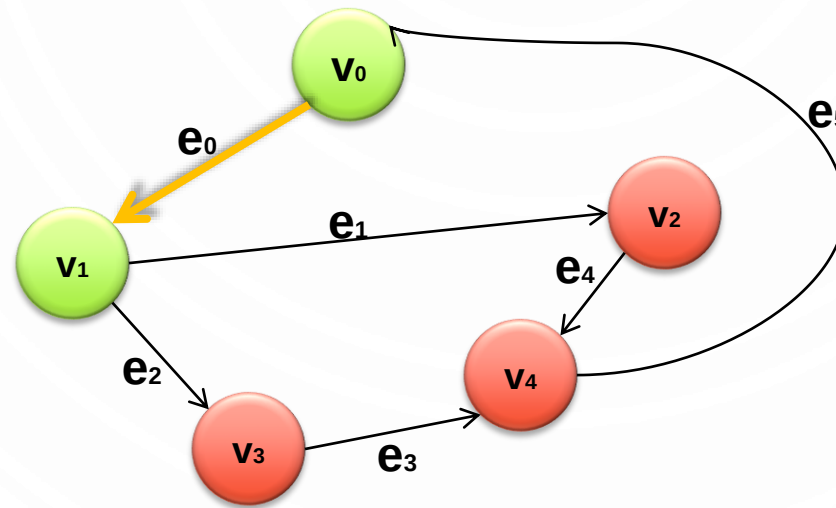
Graphs – Depth-First



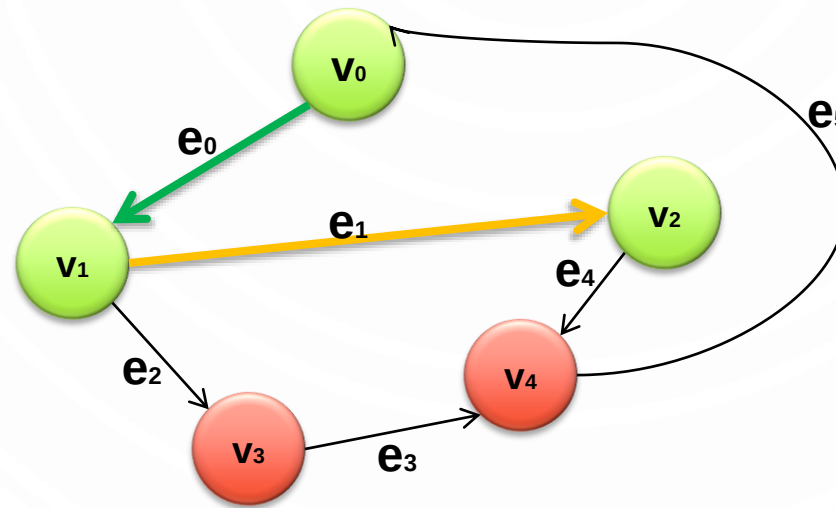
Graphs – Depth-First



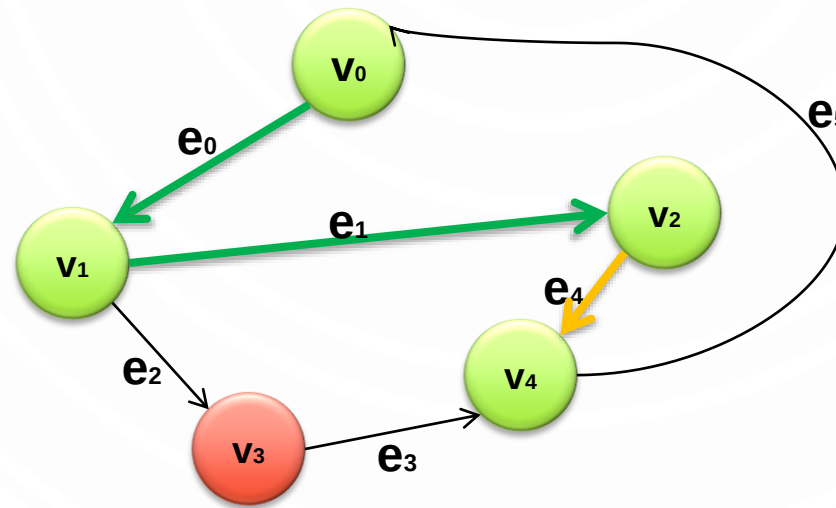
Graphs – Depth-First



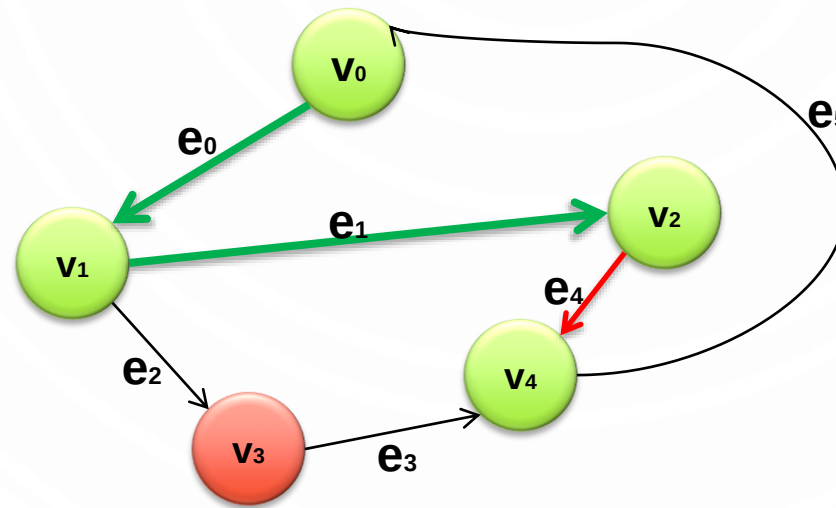
Graphs – Depth-First



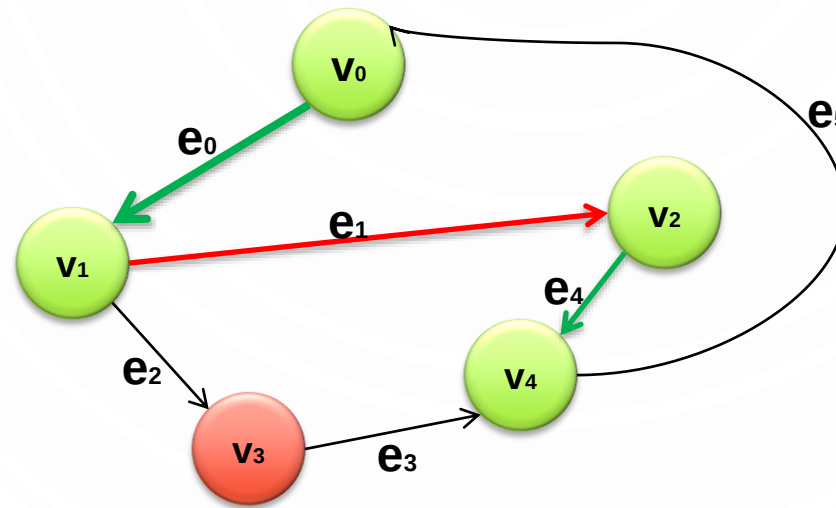
Graphs – Depth-First



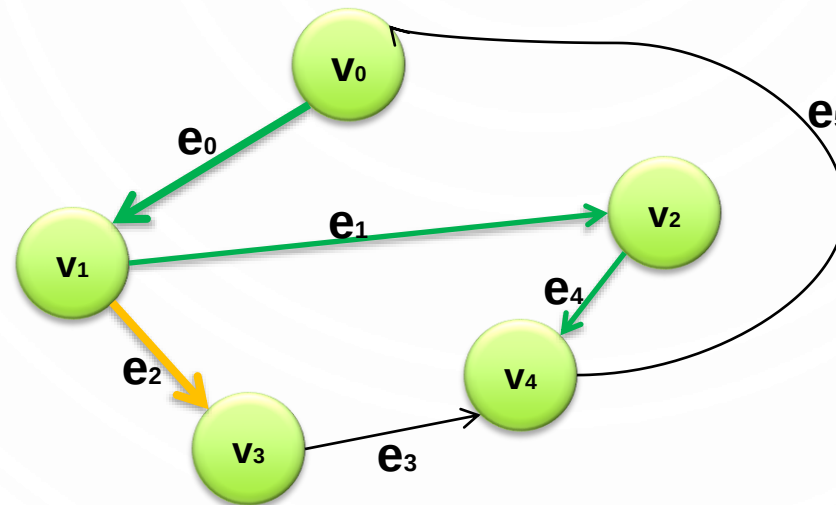
Graphs – Depth-First



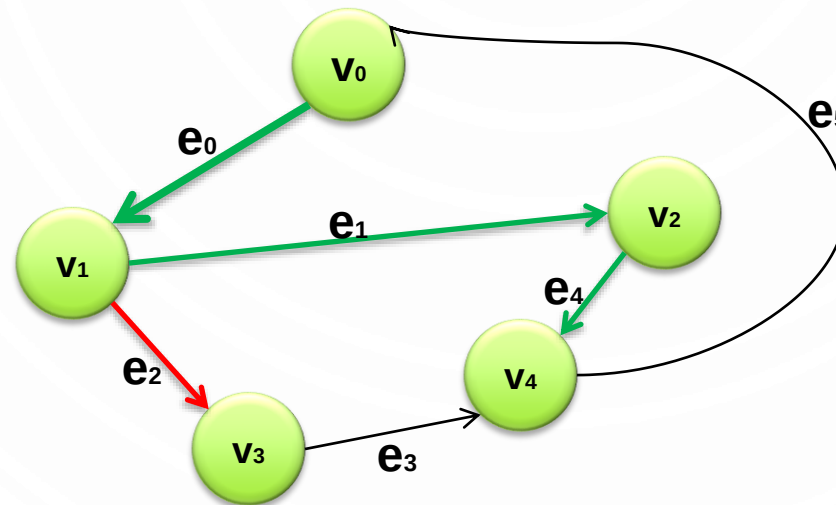
Graphs – Depth-First



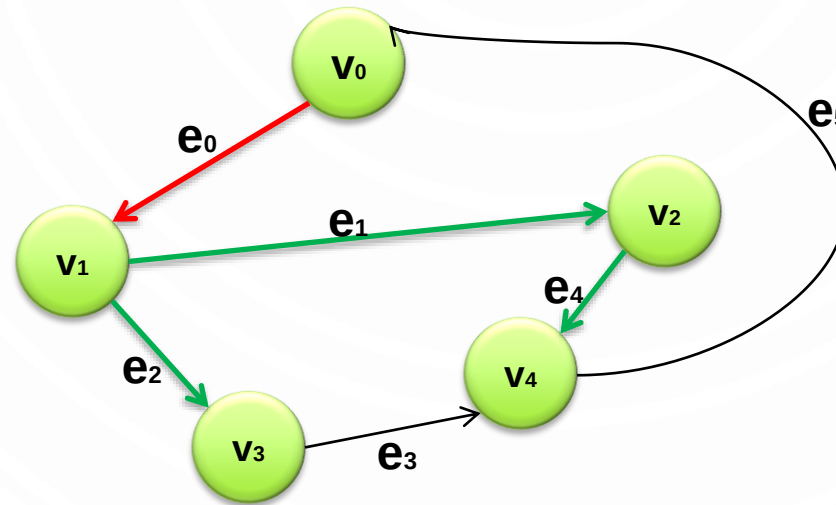
Graphs – Depth-First



Graphs – Depth-First



Graphs – Depth-First



List of visited vertices: **V_0, V_1, V_2, V_4, V_3**



Graphs – Breadth-First

- uses a queue structure for storing each node that was processed;
- checks whether a vertex has been discovered before enqueueing it to the queue;

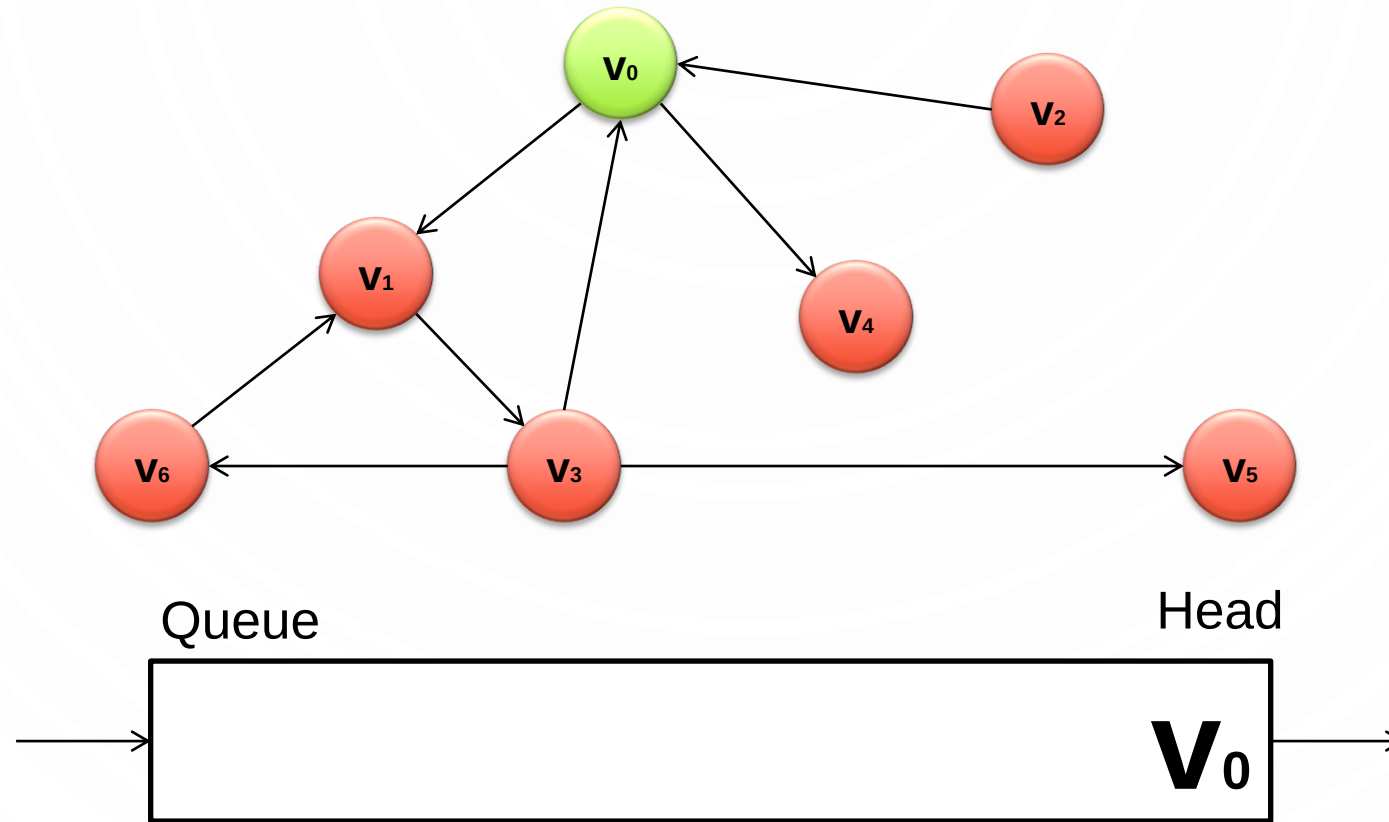


Graphs – Breadth-First

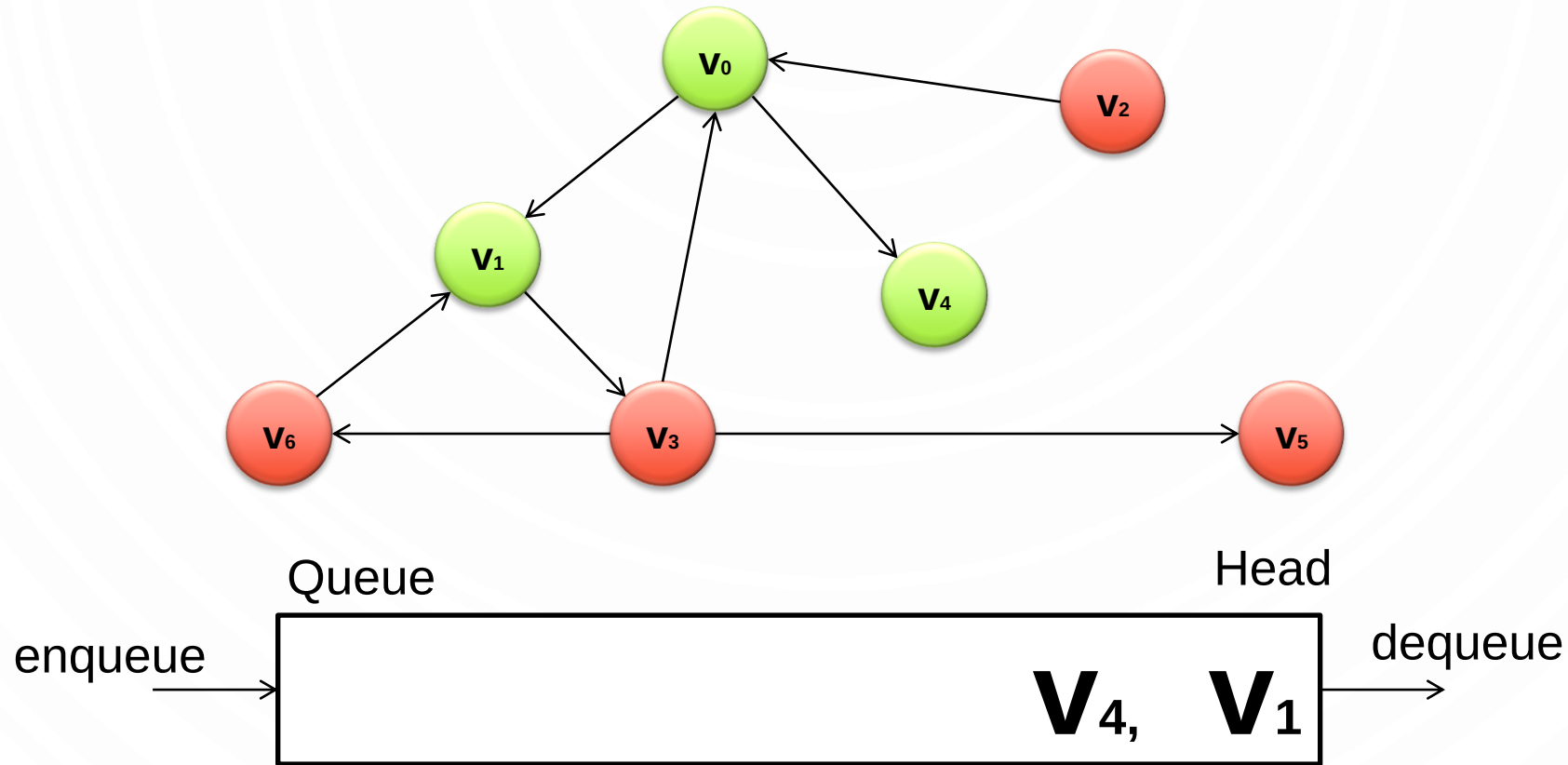
- Processing each node until the queue is empty;
- For each unlabeled vertex, u , of the current node, v :
 - processing u ;
 - label u ;
 - insert u into the queue;
- Removing vertex v from the queue;



Graphs – Breadth-First



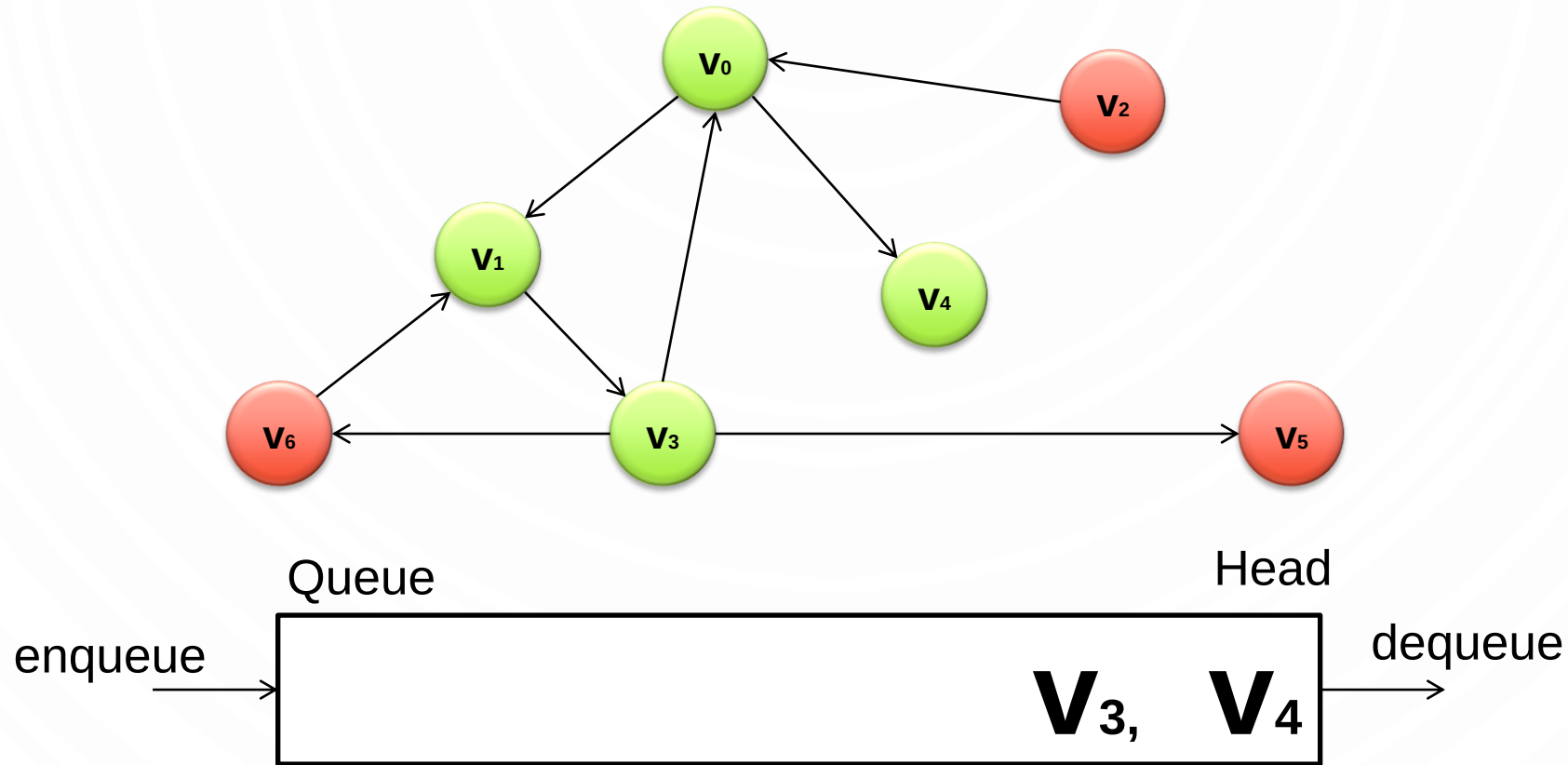
Graphs – Breadth-First



Discovered nodes: V_0 ,



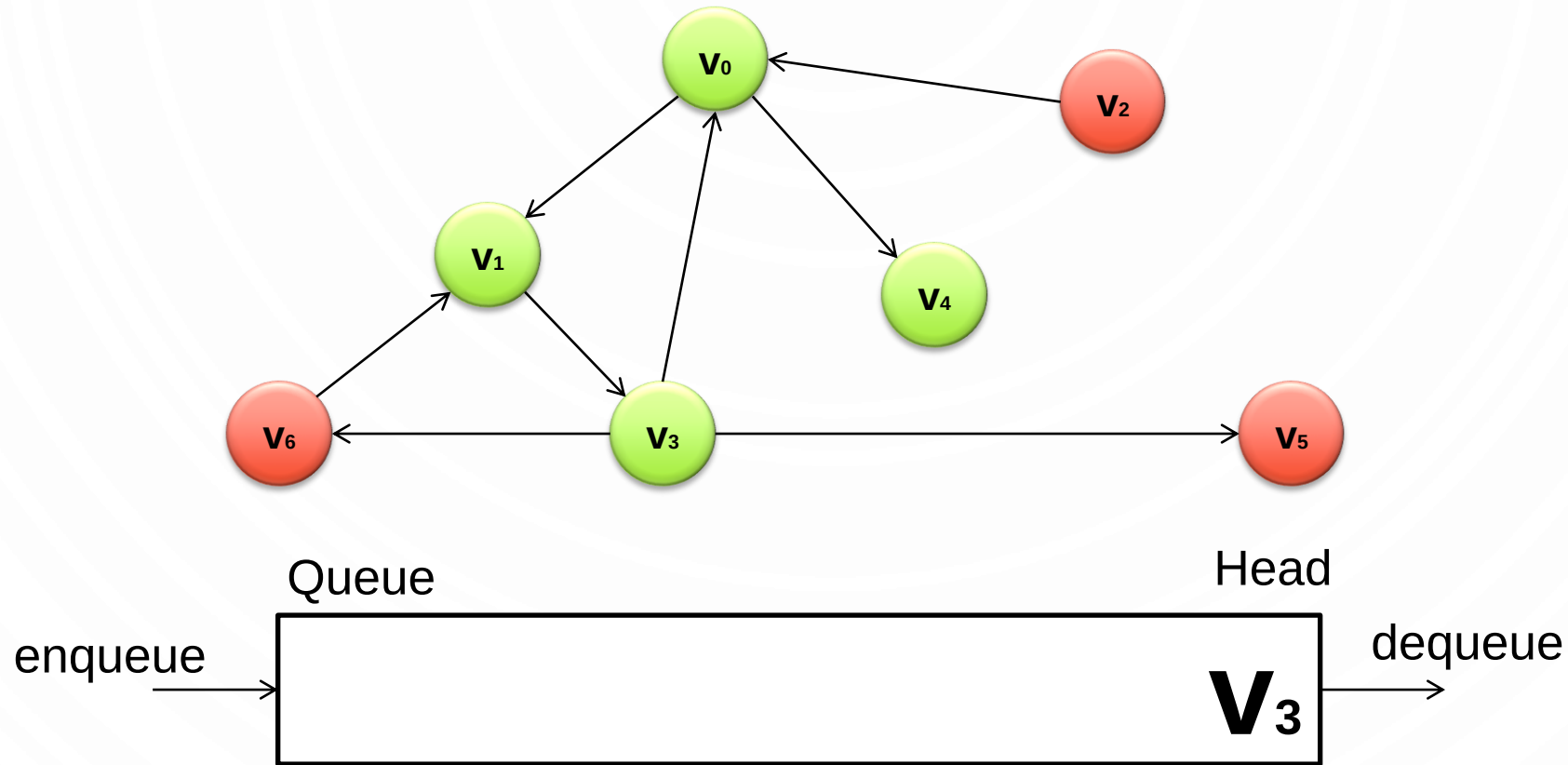
Graphs – Breadth-First



Discovered nodes : V_0 , V_1 ,



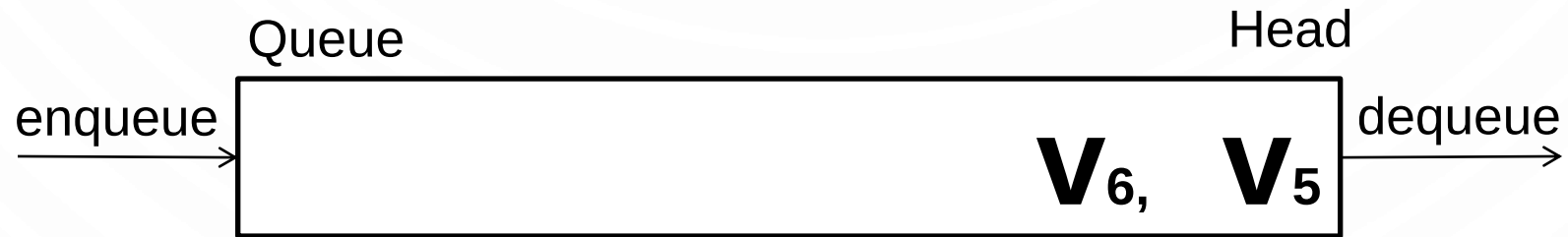
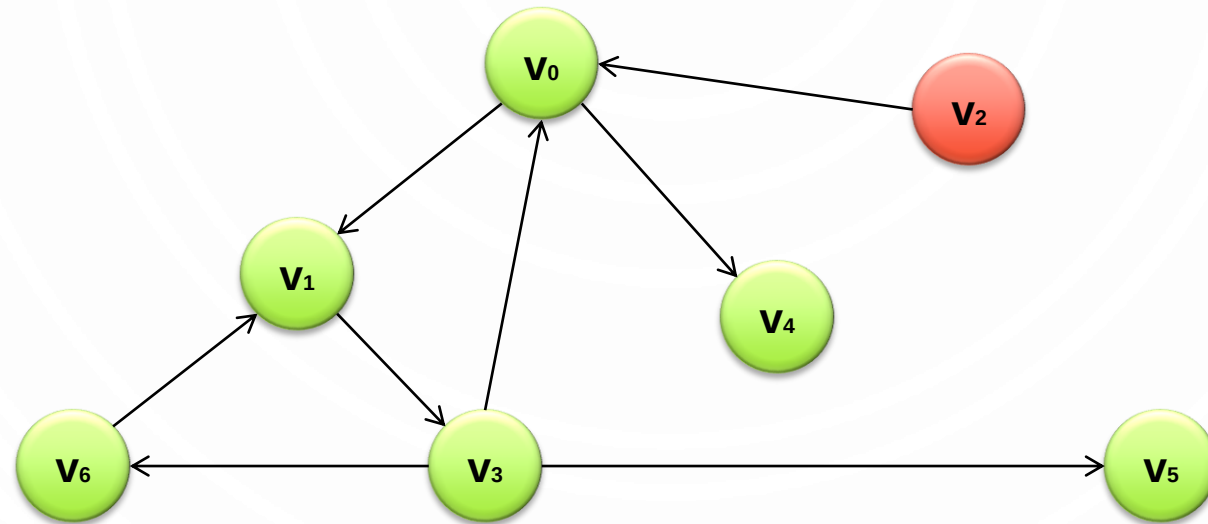
Graphs – Breadth-First



Discovered nodes : V_0 , V_1 , V_4



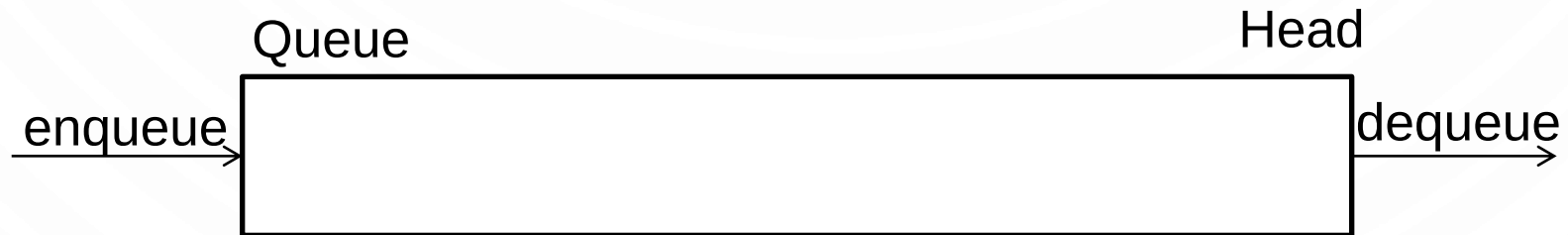
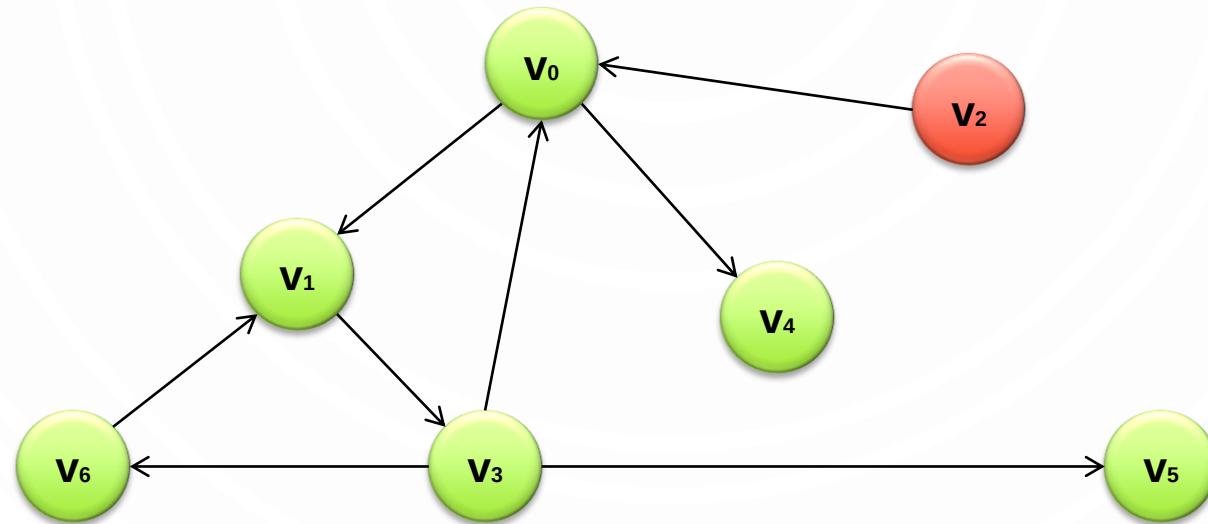
Graphs – Breadth-First



Discovered nodes : V₀, V₁, V₄, V₃



Graphs – Breadth-First



Discovered nodes : $V_0, V_1, V_4, V_3, V_5, V_6$



Metrics

McCabe complexity or the cyclomatic complexity, CmC , measures the complexity of a software program by creating the associated digraph structure;

In this graph each vertex is an instruction and edges are the path from one instruction to another:

$$CmC = nEdges - nVertices + 2nExits$$

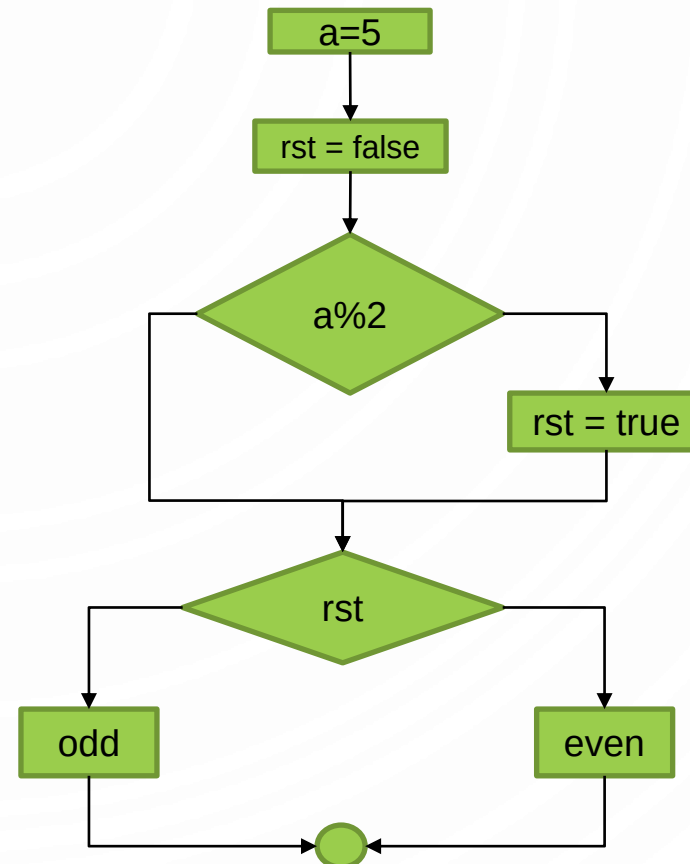
where:

- $nEdges$ – number of edges;
- $nVertices$ – number of vertices;
- $nExits$ – number of exits;



McCabe complexity

```
int a = 5;  
bool rst = false;  
if(a % 2 == 0)  
    rst = true;  
if(rst == true)  
    printf("even");  
else  
    printf("odd");
```



Software complexity

A higher level of software complexity means that ...:

- ... It's difficult to understand;
- ... There's a high probability for errors;
- ... It's harder to update/modify;
- ... A greater amount of time needed for developing;
- ... It's difficult to reuse.



Complexity impact

1-10

- Structured and well written code
- High Testability
- Cost and Effort is less

10-20

- Complex Code
- Medium Testability
- Cost and effort is Medium

20-30

- Very complex Code
- Low Testability
- Cost and Effort are high

> 40

- Not at all testable
- Very high Cost and Effort



Software complexity

```
bool isPrime() {  
    int n = 13;  
    bool prime = true;  
    for(int i=2; i<n && prime == true; i++){  
        if (n % i == 0) {  
            prime = false;    }  
    }  
    return prime;  
}
```



References

- Ion IVAN, Marius POPA, Paul POCATILU, coordonatori – *Structuri de date*, Editura ASE, Bucuresti, 2008, ISBN 978-606-505-031-0
- Ion SMEUREANU – *Programarea in limbajul C/C++*, Editura CISON, 2001
- Michael MAIN – *Data Structures & Other Objects Using Java*, Editura Pearson, 2012, ISBN: 9780132911504
- <http://www.acs.ase.ro>
- <http://www.itcsolutions.eu>

